ML-ESS

Multi-Agent AI Research Assistant

J.N. Mabiala S. Kargbo C. Akyeampong A. Akuetteh L. Bizimana

African Institute for Mathematical Sciences — Research Innovation Centre
Doctoral Training School 2026

April 2026

Manual process

- Formulate multiple search queries
- Skim dozens of web pages
- Extract relevant claims
- Resolve conflicting sources
- Draft a coherent, cited report
- Identify quality gaps

ML-ESS automates all of it

- Natural language question in
- Structured report + citations out
- Quality scores included
- Real-time progress via SSE
- REST API + web interface
- Reproducible & inspectable

Goal: replace hours of manual research with a structured, reproducible, automatically evaluated process.

A single `SharedState` object flows through four sequential agents.

Job lifecycle: `pending` → `searching` → `synthesising` → `reporting` → `evaluating` → `completed`

What it does

1. **Query generation** — LLM expands the question into N targeted search queries
2. **Web search** — Tavily (primary) → DuckDuckGo (fallback)
3. **Content scraping** — httpx + BeautifulSoup4; strips nav/footer; extracts images
4. **Evidence extraction** — LLM pulls structured *claim* ▪ *quote* ▪ *relevance* triples per source

Output written to SharedState

`search_queries` — list of strings

`sources[]`
title ▪ url ▪ snippet ▪ images

`evidence[]`
claim ▪ quote ▪ relevance ▪ source_index

Images extracted per-page are stored on each `Source` for optional embedding in the final report.

What it does

Receives all evidence fragments and asks the LLM to:

- Group fragments into coherent **themes**
- Assign a **confidence level** per theme (`high / medium / low`)
- Identify **contradictions** between sources
- Suggest a **resolution** for each contradiction

Output written to SharedState

`themes[]`
name ▪ summary ▪ evidence_indices ▪ confidence

`contradictions[]`
description ▪ evidence_indices ▪ resolution

All LLM responses are structured JSON — parsed by `chat_json()` with graceful empty-list fallback on parse failure.

Two-pass generation

Pass 1 — Outline

LLM generates section headings from the theme list: Introduction ▪ one section per theme ▪ Discussion ▪ Limitations ▪ Conclusion ▪ References.

Pass 2 — Full report

LLM writes ~1 500–2 500 words of academic Markdown with inline `[n]` citations, Mermaid diagrams, and embedded images where relevant.

Source deduplication

Sources are deduplicated by URL before building the References section — no duplicate entries regardless of how many queries hit the same page.

`report_outline` — list of section headings
`final_report` — full Markdown string

Rubric scores (0 – 1)

∗ Example: CNN vs. ViT for medical imaging demo run.

How it works

A second LLM call receives:

- The research question
- All available evidence fragments
- The final report text

It returns a scored JSON object with reasoning.

The evaluator detects unsupported claims against the evidence pool — not external ground truth.

`SharedState` is a single Pydantic v2 model every agent reads from and writes to.

```
class SharedState(BaseModel):
    research_question: str = ""

    # SearchAgent
    search_queries: list[str]       = Field
        (...)
    sources:        list[Source]    = Field
        (...)
    evidence:       list[Evidence]  = Field
        (...)

    # SynthesisAgent
    themes:         list[Theme]          = Field
        (...)
    contradictions:list[Contradiction] = Field
        (...)

    # ReportAgent
    report_outline: list[str] = Field(...)
    final_report:   str = ""

    # Evaluator
    evaluation: Optional[EvaluationScores] =
        None
```

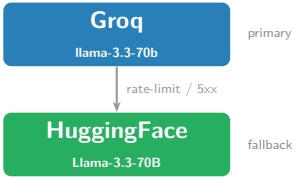
Every agent exposes exactly one function:

`run(state, on_event=None)`
$\rightarrow$ `SharedState`

No classes. No hidden side-effects. State accumulates cleanly through each stage.

`on_event(name, data)` fires structured progress events consumed by the SSE stream and job manager.

Provider strategy



Controlled by `LLM_PROVIDER=auto`. A second key `GROQ_API_KEY_2` extends rate-limit headroom.

Chat helpers (used by all agents)

```
# Plain text response
chat(client, system, user) -> str

# JSON response (parsed + fallback)
chat_json(client, system, user) -> list |
    dict
```

- Retries with backoff on 429 / 5xx
- Provider fallback built into `chat()`
- JSON fence stripping via `extract_json()`
- Empty list returned on JSON parse failure

Request lifecycle

1. `POST /api/research` — validates question, creates job
2. Returns `job_id` immediately (HTTP 202)
3. Background thread starts pipeline
4. Client polls `GET /api/research/{id}` or streams via SSE `/{id}/events`
5. Final `SharedState` persisted to SQLite

SQLite WAL mode — 3 tables

`jobs` — status, timestamps, score summary
`events` — append-only per-job event log
`states` — full `SharedState` JSON blob

In-memory write-through cache

Job objects cached in a dict for fast SSE polling — written through to SQLite on every update to survive restarts.

All `/api/research/*` endpoints require `X-API-Key` header (if `API_KEY` env var is set).

Method	Endpoint	Description
POST	`/api/research`	Submit question — returns `job_id` (202)
GET	`/api/research/{id}`	Poll status, report & evaluation scores
GET	`/api/research/{id}/events`	Server-Sent Events live progress stream
GET	`/api/research/{id}/reasoning`	Queries, sources, themes (full detail)
GET	`/api/research/{id}/pdf`	Download report as PDF
DELETE	`/api/research/{id}`	Delete a single job
GET	`/api/health`	Health check (no auth required)

```
curl -X POST http://localhost:8000/api/research \
  -H "X-API-Key: your-key" -H "Content-Type: application/json" \
  -d '{"question": "Trade-offs between CNNs and ViTs for medical imaging?"}'
```

Stack

Next.js 16 ▪ React 19 ▪ TypeScript
Tailwind CSS v4 ▪ Vercel

Real-time updates

- SSE stream via a custom `useSSE` hook
- Progress bar advances through stages
- Report rendered as Markdown on completion
- PDF download calls `/pdf` endpoint

Key components

`ResearchForm` — question input + submit
`JobStatus` — live stage indicator + spinner
`ReportView` — Markdown renderer with citations
`EvaluationCard` — score visualisation
`ReasoningPanel` — collapsible sources & themes

Set `NEXT_PUBLIC_API_URL` and `NEXT_PUBLIC_API_KEY` in `frontend/.env.local`

Layer	Technology	Role
Backend	Python 3.11, FastAPI, Uvicorn	REST API & SSE server
LLM primary	Groq — `llama-3.3-70b-versatile`	Fast inference
LLM fallback	HuggingFace — `Llama-3.3-70B-Instruct`	Rate-limit fallback
Web search	DuckDuckGo ▪ Tavily (optional)	Source discovery
Scraping	httpx ▪ BeautifulSoup4	Full-page content & images
Validation	Pydantic v2	SharedState schema
Database	SQLite WAL mode	Job & state persistence
PDF export	WeasyPrint	Markdown → styled PDF
Frontend	Next.js 16 ▪ React 19 ▪ Tailwind v4	Web UI
Hosting (FE)	Vercel	Frontend deployment
Hosting (BE)	Render	Backend API deployment

Question: *"What are the main trade-offs between CNNs and Vision Transformers for medical imaging?"*

Search

3 queries ▪ 14 sources

47 evidence fragments extracted

Synthesis

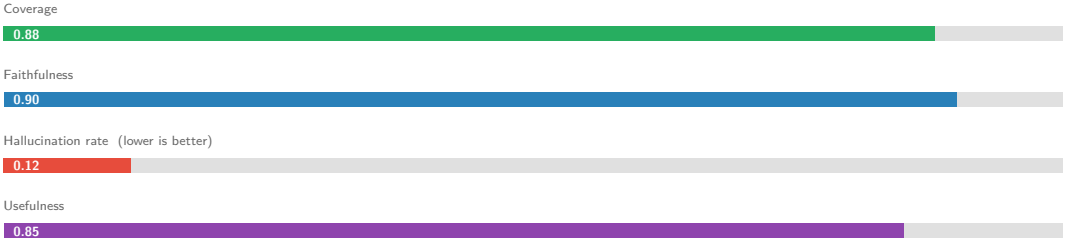
6 themes ▪ 2 contradictions

Performance ▪ Data efficiency ▪ Interpretability ▪ Computational cost ▪ Transfer learning ▪ Hybrid models

Report

~2 100 words ▪ 7 sections

Citations, 1 Mermaid diagram, 2 embedded images



Functional agents over classes

Each agent is a plain `run(state) → state` function — easy to test, swap, and trace in isolation.

Write-through cache

In-memory dict for fast SSE polling; SQLite as durable store. No cache invalidation complexity.

Two-pass report generation

Outline first, then prose — gives the LLM a structural skeleton to follow, reducing hallucination risk.

Multi-provider LLM fallback

Auto-fallback from Groq to HuggingFace prevents hard failures under rate limits during demos.

Structured JSON at every boundary

`chat_json()` enforces JSON between all agent stages — no free-form text passing.

Source deduplication by URL

Normalised before building References — no duplicate entries in the final report.

Current constraints

- **Hallucination risk** — LLM may invent claims; evaluator detects but does not prevent this
- **Open web only** — paywalled or un-indexed content unavailable
- **Sequential pipeline** — no parallel agent execution
- **Context window limits** — large evidence sets are truncated before LLM calls

Known issues

- **Search quality variance** — DDG returns fewer relevant results than Tavily
- **Evaluation circularity** — faithfulness scored against same evidence pool, not external ground truth
- **Rate limits** — free Groq tiers hit easily; HF fallback adds latency
- **PDF system libs** — WeasyPrint requires Pango/Cairo beyond pip

Pipeline

- Parallel search queries
- Iterative refinement loop
- Citation verification agent
- Domain-specific variants

Data & Quality

- Academic DB connectors (arXiv, PubMed)
- External fact-checking
- Human-in-the-loop approval
- Fine-tuned smaller models

Platform

- WebSocket real-time updates
- Multi-user job isolation
- Scheduled recurring jobs
- WhatsApp / email delivery

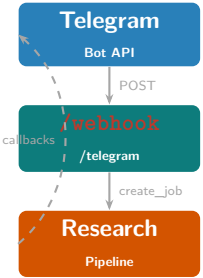
Architecture is designed for incremental extension: add an agent module → register fields in `SharedState` → insert one call in `pipeline.py`.

How it works

1. User sends a research question to the bot
2. Bot acknowledges and starts a background job
3. Progress updates sent at each pipeline stage
4. Final report delivered as chunked messages
5. Quality scores included in the summary

No business verification needed — bot token from @BotFather is instant.

Architecture



Reuses existing callback pattern
`on_progress` + `on_complete`
Same as WhatsApp integration

Backend — Render

- Python/FastAPI service on Render free tier
- Auto-deploys from `main` branch on push
- Persistent SQLite volume for job history
- Health check on `/api/health`

```
# Set in Render > Environment
GROQ_API_KEY=gsk_...
GROQ_API_KEY_2=gsk_...
HF_API_KEY=hf_...
TAVILY_API_KEY=tvly_...
API_KEY=your-secret
LLM_PROVIDER=auto
```

Frontend — Vercel

- Next.js 16 App Router deployed to Vercel
- Points to the Render backend URL
- Zero-config CI/CD from Git

Vercel env vars

`NEXT_PUBLIC_API_URL`
— Render service URL

`NEXT_PUBLIC_API_KEY`
— matches backend `API_KEY`

What we built

- End-to-end automated research pipeline with 4 specialised AI agents
- FastAPI backend: async jobs, SSE streaming, SQLite persistence
- Next.js frontend with real-time progress and PDF export
- Telegram bot for conversational access
- Multi-provider LLM layer with automatic fallback and retries
- Built-in self-evaluation on a 4-criterion quality rubric

Key takeaways

- Multi-agent decomposition makes complex tasks tractable and inspectable
- Structured JSON at every boundary is essential for reliability
- Self-evaluation adds accountability without external ground truth
- Provider abstraction is critical for demo reliability under rate limits

Every component is inspectable — from a single question to a scored, cited report.

Thank you

Questions?

J.N. Mabiala ▪ S. Kargbo ▪ C. Akyeampong ▪ A. Nunoo ▪ Lambert

AIMS RIC ▪ Doctoral Training School 2026

FastAPI ▪ Groq / Llama-3.3-70B ▪ Next.js 16 ▪ Telegram Bot ▪ Pydantic v2 ▪ SQLite WAL ▪ Render ▪ Vercel